Docker 数据卷与持久化

第 9 讲

目标

让容器的"易销毁"也不再丢数据

- 数据痛点: 可写层 **随容器销毁而消失**
- 三种方式: Volume / Bind Mount / tmpfs
- 数据卷命令: `create / ls / inspect / rm`
- **热开发**: 挂源码目录, 改代码即时生效

一、数据为什么不持久

> 可写层随容器没了

容器删了，写在里面的数据也一起没了

```
docker run -d --name web -p 8080:80 nginx
docker exec -it web bash \
  -c 'echo hello > /usr/share/nginx/html/a.txt'
docker rm -f web          # 删容器
docker run -d --name web2 -p 8080:80 nginx
curl localhost:8080/a.txt  # 404! a.txt 没了
```



> 可写层 = 容器专属，容器没了它就没了

三招把数据"请出"可写层

方式	挂载源	用途
Volume	Docker管理的位置	DB、有状态数据
Bind Mount	宿主机任意目录	开发调试、传配置
tmpfs	内存	临时敏感数据

> 共同点: 把宿主机侧的存储, mount 进容器某个目录

二、数据卷 Volume

> Docker 管的"外置硬盘"

volume 是独立于容器的外部存储

```
docker volume create mydata     # 建卷
docker volume ls                # 列卷
docker volume inspect mydata    # 看细节(Mountpoint)
docker volume rm mydata         # 删卷
```



> 容器删除后卷还在, 挂上同一卷数据就回来

把卷挂进容器: -v 或 --mount

```
docker run -d --name web \
  -v webdata:/usr/share/nginx/html nginx
# 等价、更清晰:
docker run -d --name web \
  --mount source=webdata,target=/usr/share/nginx/html nginx
```



> 卷不存在会自动建; 生产更推荐 --mount

三、绑定挂载 Bind Mount

> 直接把宿主目录"借"给容器

bind mount 实时同步宿主与容器

```
docker run -d --name web \
  -v "$PWD/html":/usr/share/nginx/html nginx
# --mount 写法:
docker run -d --name web \
  --mount type=bind,source="$PWD/html",target=/usr/share/nginx/html nginx
```



关键: 必须是**绝对路径**; 改宿主文件, 容器立刻同步

四、共享与热开发

> 一卷多用, 一挂即改

多容器挂同一卷 = 共享

```
docker volume create webdata
docker run -d --name site1 -v webdata:/usr/share/nginx/html -p 8081:80 nginx
docker run -d --name site2 -v webdata:/usr/share/nginx/html -p 8082:80 nginx

docker exec -it site1 bash -c 'echo shared > …/x.html'
curl localhost:8082/x.html    # site2 也看到 shared!
```



> 一份数据, 多个容器共享读写

热开发: 源码目录挂进去, 改代码即时生效

```
docker run -d -p 8000:8000 \
  -v "$PWD/app":/app \
  flask-app python app.py --reload
```



> 改 .py → 刷新页面即生效, 免反复 docker build

只读挂载, 防止容器内误改

```
docker run -d -v "$PWD/conf":/etc/nginx/conf.d:ro nginx
```

> `:ro` 让容器只能读, 宿主机配置不被容器碰坏

Volume 还是 Bind Mount?

维度	Volume	Bind Mount
管理	Docker管	宿主任意路径
可见性	隐藏在 /var/lib/docker	直接可见
适用	生产、有状态DB	**开发调试、直改代码**

> 拿不准用 Volume; 要实时改代码用 Bind Mount

五、实战一

数据卷让数据跨容器活

> 删容器, 卷还在

删了重建, 数据被卷留下来

```
docker volume create webdata
docker run -d --name site1 -v webdata:/usr/share/nginx/html -p 8081:80 nginx
docker exec -it site1 bash -c 'echo persist > …/p.txt'
docker rm -f site1                          # 容器没了

docker run -d --name site2 -v webdata:/usr/share/nginx/html -p 8082:80 nginx
curl localhost:8082/p.txt                   # persist 还在!
docker volume rm webdata                    # 清理
```



六、实战二

bind mount 做热开发

> 改宿主, 容器即变

改一个文件, curl 立刻变

```
mkdir -p ~/dev/demo && echo '<h1>Demo</h1>' > ~/dev/demo/index.html

docker run -d --name dev -p 8080:80 \
  -v "$HOME/dev/demo":/usr/share/nginx/html nginx
curl localhost:8080            # Demo

echo '<h1>Updated!</h1>' > ~/dev/demo/index.html
curl localhost:8080            # Updated! 无 build
docker exec -it dev cat /usr/share/nginx/html/index.html  # 容器内同步
```



一张表带走全部要点

方面	要点
痛点	可写层随容器销毁而消失
三种方式	Volume / Bind Mount / tmpfs
数据卷	create/ls/inspect/rm · `-v 卷:路径` · `--mount`
绑定挂载	绝对路径 · 实时双向同步 · `:ro` 只读
共享	多容器挂同一卷/同一路径
热开发	挂源码目录 → 改代码即时生效

数据留得住, 代码热更新 – 容器终于能"过日子"了

> 下一步: 部署编程语言(C/C++、Java)。Dockerfile 把源码+环境打成镜像, 用掉前面全部功夫

谢谢大家

Q & A